
INTRO TO ALGORITHMS ENGINEERING

Course Project

Project 20 - Eigenvector Centrality

Author

Ayush Subham Moharana
2024101074

Ishaan Shiv Kumar
2024101098

Contents

1	Introduction	1
2	Objective	1
3	Background and Related Work	1
4	Problem Statement	2
5	Mathematical Formulation	3
6	Node-Centrality and Normalization	4
7	Algorithms Studied	4
7.1	Direct Eigenvalue Decomposition	4
7.2	Power Iteration Method	5
8	Implementation Details	7
9	Results and Interpretation	8
9.1	Runtime Comparison	8
9.2	Accuracy Analysis	9
9.3	Key Insight	10
10	Application to Molecular Graphs	11
10.1	Graph Representation of Alkanes	11
10.2	Case Study: Butane Isomers	11
10.3	Extension to Larger Molecules	12
10.4	Structural Insight	12
10.5	Connection to Node-Centrality Theory	12
10.6	Eigenvector-Based Topological Indices	13
10.7	Visualization of Topological Indices	13
10.8	Visualization of Centrality Profiles	14
10.9	Correlation with Physical Properties	14
10.10	Relation to Project Goals	15
11	Tensor Extension of Centrality	16
12	Conclusion	16

1 Introduction

In graph theory, centrality measures are used to quantify the relative importance of nodes within a network. Identifying important nodes is a fundamental problem in network analysis, with applications spanning social networks, transportation systems, communication networks, biological systems, epidemiology, and recommendation systems. The significance of a node depends on the structure of the network and the nature of interactions it represents.

Among the various centrality measures, eigenvector centrality is especially significant because it evaluates not only the number of connections a node has, but also the importance of the nodes to which it is connected. This yields a more meaningful global measure of influence compared to simpler local measures such as degree centrality.

Eigenvector centrality is grounded in the spectral properties of the graph and is obtained from the dominant eigenvector of the adjacency matrix. While direct matrix-based methods suffice for small graphs, they become prohibitively expensive in time and memory as network size grows. This project studies two computational approaches to address this challenge: direct eigenvalue decomposition, which serves as an exact baseline, and the Power Iteration method, a scalable iterative alternative suited to large and sparse networks.

2 Objective

This project pursues the following objectives:

1. Study the mathematical foundation of eigenvector centrality within the framework of spectral graph theory.
2. Implement direct eigenvalue decomposition as an exact baseline method for computing eigenvector centrality.
3. Implement the Power Iteration method as a scalable iterative alternative suitable for large and sparse networks.
4. Analyze and compare the correctness, convergence behavior, and computational efficiency of both methods across graphs of varying sizes and structures.
5. Validate the obtained results against the reference implementation of eigenvector centrality provided by the NetworkX library.

3 Background and Related Work

Centrality analysis is a fundamental topic in graph theory and network science, where the objective is to identify the most important or influential nodes in a network. The concept of importance varies depending on the application domain: in social networks, important nodes may represent influential individuals; in transportation networks, critical intersections; in biological systems, essential proteins or genes; and in communication networks, strategically important routers.

Several centrality measures have been developed, each based on a different interpretation of structural significance. Degree centrality counts direct connections and serves as the

simplest local measure. Closeness centrality quantifies how efficiently a node can reach all others via shortest paths. Betweenness centrality measures how frequently a node lies on shortest paths between other pairs of nodes, identifying bridges between communities. Each of these measures, however, considers only the immediate or path-based neighborhood of a node and does not account for the global propagation of influence.

Eigenvector centrality addresses this limitation by assigning each node a score proportional to the sum of its neighbors' scores. This recursive definition means that a connection to a highly influential node contributes more than a connection to a peripheral one. As a result, eigenvector centrality captures global influence rather than purely local structure, making it well suited to applications such as social influence analysis, citation networks, and ranking problems.

A closely related algorithm is PageRank, developed for ranking pages in web search. PageRank can be viewed as a generalization of eigenvector centrality to directed graphs, augmented with a damping factor that models the probability of a random surfer following a link versus jumping to an arbitrary page. This modification improves numerical stability and ensures convergence even in graphs with dangling nodes or disconnected components. Both eigenvector centrality and PageRank rest on the same underlying principle: importance propagates through connections, and a node inherits influence from its neighbors.

From a theoretical standpoint, eigenvector centrality belongs to the field of spectral graph theory, in which structural properties of graphs are studied through the eigenvalues and eigenvectors of associated matrices. Two matrices are central to this field: the adjacency matrix, whose dominant eigenvector yields eigenvector centrality scores, and the graph Laplacian, which is more commonly used in clustering and diffusion analysis. Spectral methods provide mathematically rigorous tools for network analysis and underpin a wide range of algorithms in ranking, community detection, and network optimization.

Practical computation of eigenvector centrality has been studied extensively. Direct eigendecomposition yields an exact solution but scales poorly with network size. Iterative methods, particularly Power Iteration, are preferred for large and sparse networks due to their lower memory footprint and computational cost. The NetworkX library provides a well-tested reference implementation of eigenvector centrality using Power Iteration, which is used in this project as a benchmark for correctness and performance comparison. The following sections formalize the computational problem and examine both approaches in detail.

4 Problem Statement

Given an undirected, connected graph $G = (V, E)$ with adjacency matrix $A \in \mathbb{R}^{n \times n}$, the problem is to compute a centrality vector $\mathbf{x} \in \mathbb{R}^n$ satisfying

$$A\mathbf{x} = \lambda_1\mathbf{x},$$

where λ_1 is the dominant eigenvalue of A . Each component x_i represents the eigenvector centrality score of node $i \in V$.

The method must satisfy the following constraints:

- The graph is undirected and connected, ensuring that λ_1 is real, positive, and unique by the Perron–Frobenius theorem

- The implementation must remain applicable to sparse graphs with n up to [your largest tested graph size]. [FIX]
- The computed centrality vector must converge to within a tolerance of $\varepsilon =$ [your tolerance value] of the NetworkX reference solution under the L_1 norm.

A solution is considered successful when both methods produce centrality rankings consistent with the reference implementation and their runtime and convergence behavior have been characterized across graphs of varying size and structure.

5 Mathematical Formulation

Let $G = (V, E)$ be an undirected, connected graph with $|V| = n$ vertices. The structure of G is encoded in its adjacency matrix $A = (a_{ij}) \in \mathbb{R}^{n \times n}$, defined as

$$a_{ij} = \begin{cases} 1, & \text{if } (i, j) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

Since G is undirected, A is symmetric, and all eigenvalues of A are real. Eigenvector centrality assigns a score x_i to each node $i \in V$ proportional to the sum of the scores of its neighbors:

$$x_i = \frac{1}{\lambda} \sum_{j=1}^n a_{ij} x_j.$$

In matrix form, this is the standard eigenvalue equation

$$A\mathbf{x} = \lambda\mathbf{x}, \quad \mathbf{x} \in \mathbb{R}^n.$$

The centrality vector is taken as the eigenvector corresponding to the largest eigenvalue λ_1 , as this choice guarantees non-negative components suitable for representing importance scores. This guarantee follows from the Perron–Frobenius theorem: for a connected graph with a non-negative adjacency matrix, λ_1 is real, positive, and strictly greater than all other eigenvalues, and its corresponding eigenvector has strictly positive components. Since eigenvectors are defined only up to a scalar multiple, $\mathbf{x}$ must be normalized after computation. In this project, two normalization schemes are considered for different purposes.

For computational validation and comparison with the NetworkX reference implementation, the L_1 normalization is used:

$$\sum_{i=1}^n x_i = 1.$$

However, from a theoretical standpoint, eigenvector centrality satisfies the conditions of a valid node-centrality only under Euclidean (L_2) normalization. In this case, the centrality vector is scaled as:

$$nc_i = \sqrt{2} \cdot x_i$$

which ensures that the maximum value of 1 is uniquely attained by the center of a star graph. This distinction is important: L_1 normalization is used for computational convenience and benchmarking, while L_2 normalization is required for theoretical correctness in the context of node-centrality.

6 Node-Centrality and Normalization

A key theoretical question explored in this project is whether eigenvector centrality qualifies as a valid *node-centrality*. According to Ruhnau (2000), a centrality measure must satisfy:

- Values lie in the interval $[0, 1]$
- The value 1 is achieved **only** by the center of a star graph

Eigenvector centrality depends critically on how the eigenvector is normalized. Three common normalization methods are:

- Maximum norm (L_∞): produces relative centrality within a graph
- Sum norm (L_1): produces proportional centrality
- Euclidean norm (L_2): produces a globally comparable measure

Only Euclidean normalization satisfies the node-centrality conditions. The normalized centrality is given by:

$$nc_i = \sqrt{2} \cdot x_i$$

This ensures that the maximum possible value is 1 and is uniquely attained by the center of a star graph.

7 Algorithms Studied

Two methods are studied for computing eigenvector centrality: Direct Eigenvalue Decomposition, a naive approach with high computational cost, and the Power Iteration Method, a more scalable iterative alternative.

7.1 Direct Eigenvalue Decomposition

Direct Eigenvalue Decomposition computes eigenvector centrality by applying full matrix eigendecomposition to the adjacency matrix A . The procedure constructs A explicitly, computes all eigenvalues and eigenvectors, selects the eigenvector corresponding to the largest eigenvalue, and normalizes the result using L_1 normalization.

Pseudocode

Input:

Graph $G = (V, E)$

Output:

Eigenvector centrality vector x

1. Construct adjacency matrix A from G
2. Compute all eigenvalues (λ) and eigenvectors X of A
3. Find index k such that $\lambda_k = \max(\lambda)$
4. Select eigenvector $x = X[:, k]$
5. Normalize x using L1 normalization $x = x / \text{sum}(x)$
6. Return x

The time complexity of this method is

$$O(n^3)$$

where (n) is the number of vertices in the graph. This is because full matrix eigen-decomposition requires cubic time complexity for dense matrices. The space complexity is

$$O(n^2)$$

due to the storage requirement of the adjacency matrix.

The primary advantage of this method is that it yields an exact solution. Its main limitation is poor scalability: both runtime and memory usage grow rapidly with graph size, making it impractical for large sparse networks.

7.2 Power Iteration Method

Power Iteration approximates the dominant eigenvector of A by starting from an initial non-zero vector and repeatedly multiplying by A , normalizing at each step to prevent unbounded growth. The iterative update rule is

$$x^{(k+1)} = \frac{Ax^{(k)}}{\|Ax^{(k)}\|_2}.$$

Convergence is guaranteed when the spectral gap is strictly positive, i.e., $|\lambda_1| > |\lambda_2|$, where λ_1 and λ_2 are the two largest eigenvalues in magnitude. A larger spectral gap yields faster convergence. The algorithm terminates when

$$\|x^{(k+1)} - x^{(k)}\|_1 < \varepsilon,$$

where ε is a predefined tolerance, or when a maximum number of iterations is reached.

Pseudocode

Input:

Graph $G = (V, E)$

Maximum iterations `max_iter`

Tolerance `epsilon`

Output:

Eigenvector centrality vector x

1. Construct adjacency matrix A from G

2. Initialize vector x with equal values
and L2 normalization

$x = [1/\sqrt{n}, 1/\sqrt{n}, \dots, 1/\sqrt{n}]$

3. Repeat for $k = 1$ to `max_iter`

$x_{\text{new}} = A * x$

Normalize x_{new} using L2 norm

$x_{\text{new}} = x_{\text{new}} / ||x_{\text{new}}||_2$

If $||x_{\text{new}} - x|| < \text{epsilon}$

 Normalize x_{new} using L1 norm

$x_{\text{new}} = x_{\text{new}} / \text{sum}(x_{\text{new}})$

 Return x_{new}

$x = x_{\text{new}}$

4. If maximum iterations are reached

 Normalize x using L1 norm

$x = x / \text{sum}(x)$

5. Return x

The time complexity of each iteration is

$$O(m)$$

where m is the number of edges in the graph. If the algorithm converges in k iterations, the total time complexity becomes

$$O(mk)$$

which is significantly more efficient than direct eigendecomposition for sparse graphs. The space complexity is approximately

$$O(m)$$

when sparse matrix representations are used, making the method memory efficient for large networks.

The main advantage of Power Iteration is scalability: it is simple to implement, memory efficient, and well suited to large sparse networks. Its convergence rate depends on the spectral gap; a small gap between λ_1 and λ_2 can result in slow convergence.

8 Implementation Details

The project was implemented in C++ using an adjacency-list representation for efficient storage and traversal of sparse graphs. Two separate modules were implemented, one for each computational method.

Direct Eigenvalue Decomposition was implemented using the Eigen C++ linear algebra library. The adjacency matrix was constructed explicitly as a dense matrix, and the `SelfAdjointEigenSolver` class was used to perform full eigendecomposition. Since the graph is undirected and A is symmetric, all eigenvalues are real and the dominant eigenvector corresponds to the largest eigenvalue.

Power Iteration was implemented manually using iterative matrix-vector multiplication over the adjacency list. At each iteration, the vector was normalized using the L_2 norm for numerical stability. Upon convergence, L_1 normalization was applied to the final vector to ensure consistency with the Direct method for comparison purposes. The stopping criterion was

$$\|x^{(k+1)} - x^{(k)}\|_1 < 10^{-6},$$

with a maximum iteration limit of 100.

To evaluate both methods, synthetic graphs were generated using the Erdős–Rényi random graph model. Graph sizes ranging from 100 to 1000 vertices were tested in increments of 100, with 5 randomly generated instances per size using a fixed seed for reproducibility, yielding 50 graphs in total. For each graph, both methods were executed independently on the same input. Runtime was measured using high-resolution clock timing in C++, and the mean runtime across the 5 instances was recorded per graph size.

Accuracy was evaluated by comparing the Power Iteration result against the Direct Eigenvalue Decomposition result using the percentage relative error:

$$\text{Error (\%)} = \frac{\|x^{(\text{power})} - x^{(\text{direct})}\|_2}{\|x^{(\text{direct})}\|_2} \times 100.$$

Python scripts were written to automate dataset generation, experiment execution, result aggregation, and plot generation, ensuring full reproducibility of the experimental pipeline.

9 Results and Interpretation

This section presents the empirical evaluation of the two approaches:

- Power Iteration (iterative method)
- Direct Eigenvalue Decomposition (exact method)

9.1 Runtime Comparison

Vertices	Power Iteration (ms)	Direct Method (ms)
100	0.1266	1.6246
200	0.3509	17.4663
300	0.4743	59.7081
400	0.5003	121.9982
500	0.4557	236.8484
600	0.5539	418.6962
700	0.5355	654.3904
800	0.4523	954.2556
900	0.5714	1350.904
1000	0.6316	1836.648

Table 1: Runtime comparison between Power Iteration and Direct Eigendecomposition

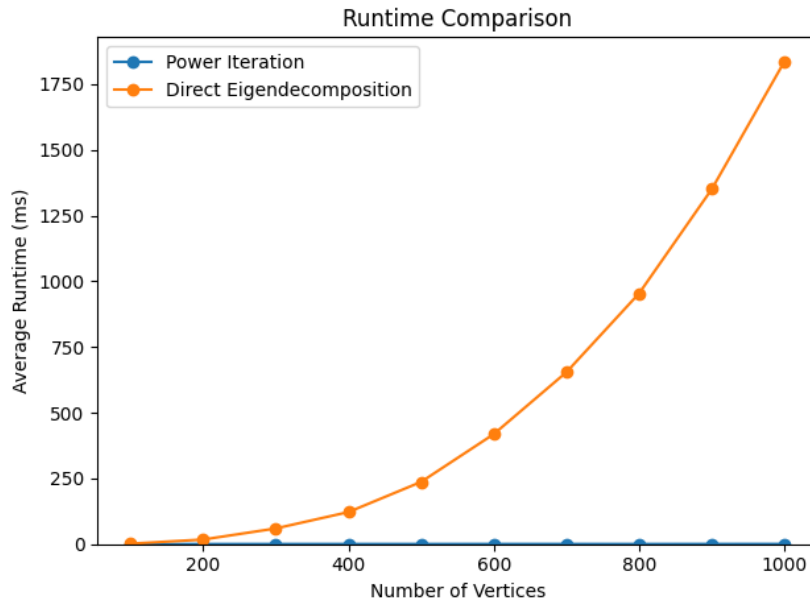


Figure 1: Runtime comparison of Power Iteration vs Direct Eigendecomposition

9.1.1 Observation

- Power Iteration runtime remains nearly constant and very low
- Direct eigendecomposition grows rapidly with graph size
- The direct method shows approximately cubic growth ($O(n^3)$)
- Power Iteration scales significantly better for large graphs

9.2 Accuracy Analysis

Vertices	Relative Error (%)
100	10.2693
200	0.5548
300	0.000131
400	3.85×10^{-5}
500	4.84×10^{-5}
600	4.14×10^{-5}
700	5.43×10^{-5}
800	3.83×10^{-5}
900	3.55×10^{-5}
1000	2.43×10^{-5}

Table 2: Relative Error between Power Iteration and Direct Method

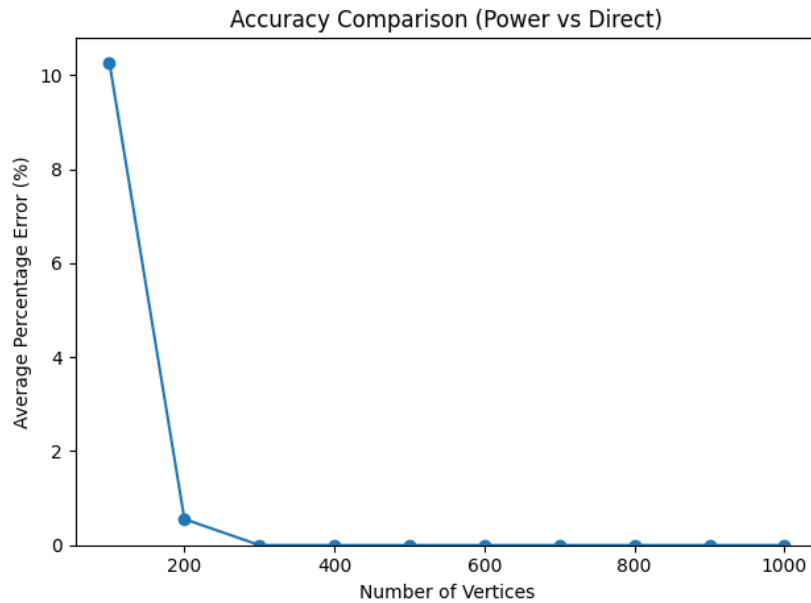


Figure 2: Accuracy comparison between Power Iteration and Direct Method

9.2.1 Observation

- Error is high for very small graphs ($n = 100$)

- Error rapidly decreases as graph size increases
- For $n \geq 300$, error becomes negligible ($< 10^{-4}$)
- Power Iteration converges very closely to the exact solution

9.3 Key Insight

- Power Iteration provides an excellent trade-off between speed and accuracy
- For large graphs, it achieves near-identical results to eigendecomposition
- Direct eigendecomposition is computationally expensive and impractical for large n

10 Application to Molecular Graphs

In this project, molecular structures are modeled as undirected graphs, where:

- Carbon atoms are represented as vertices
- Bonds between carbon atoms are represented as edges
- Hydrogen atoms are omitted, resulting in a skeletal graph

This abstraction preserves the connectivity and structural features of the molecule while simplifying analysis.

10.1 Graph Representation of Alkanes

Alkanes form an ideal class of molecules for analysis since they consist of only single bonds and have well-defined structural isomers.

Two fundamental structural patterns are observed:

- **Linear chains** (e.g., n-butane, n-pentane)
- **Branched structures** (e.g., isobutane, isopentane, neopentane)

These structures differ significantly in how connectivity is distributed across atoms.

10.2 Case Study: Butane Isomers

- **n-butane**: a linear chain $C_1 - C_2 - C_3 - C_4$
- **isobutane**: a star-like structure with one central carbon (degree 3)

From computed results:

- n-butane centralities: (0.372, 0.602, 0.602, 0.372)
- isobutane centralities: (0.408, 0.707, 0.408, 0.408)

10.2.1 Interpretation

- In n-butane, centrality is distributed across the chain
- In isobutane, one node dominates (central hub)
- The branched structure behaves similarly to a star graph

This confirms that eigenvector centrality captures global structural importance.

10.3 Extension to Larger Molecules

For larger molecules such as pentane and hexane:

- Linear molecules show symmetric centrality profiles
- Central atoms attain the highest values
- End atoms have the lowest centrality

Example (n-pentane):

$$(0.289, 0.500, 0.577, 0.500, 0.289)$$

In contrast, branched isomers concentrate centrality:

- isopentane: highest value at branching carbon
- neopentane: central atom achieves maximum centrality

10.4 Structural Insight

The results demonstrate a key property:

- **Linear chains** $\rightarrow$ distributed importance
- **Branched structures** $\rightarrow$ concentrated importance

This is consistent with spectral graph theory:

- Nodes with higher degree and better connectivity dominate eigenvectors
- Branching increases influence concentration

10.5 Connection to Node-Centrality Theory

Using Euclidean normalization:

$$nc_i = \sqrt{2} \cdot x_i$$

the central atom in star-like structures (e.g., isobutane, neopentane) approaches the maximum value of 1.

This aligns with the theoretical requirement that:

- Only the center of a star graph attains maximum centrality

Thus, molecular graphs provide a natural validation of node-centrality theory.

10.6 Eigenvector-Based Topological Indices

Using eigenvector centrality values x_i , several topological indices are defined:

$$\begin{aligned}
 M1_x &= \sum_{(i,j) \in E} (x_i + x_j) \\
 M2_x &= \sum_{(i,j) \in E} x_i x_j \\
 F_x &= \sum_{(i,j) \in E} (x_i^2 + x_j^2) \\
 H_x &= \sum_{(i,j) \in E} \frac{2}{x_i + x_j} \\
 IS_x &= \sum_{(i,j) \in E} \frac{x_i x_j}{x_i + x_j} \\
 RR_x &= \sum_{(i,j) \in E} \sqrt{x_i x_j} \\
 ABC_x &= \sum_{(i,j) \in E} \sqrt{\frac{x_i + x_j - 2}{x_i x_j}}
 \end{aligned}$$

These indices summarize the global structure of the molecule into scalar values.

10.7 Visualization of Topological Indices

The computed topological indices for various alkane isomers are shown in Figure 3. These values reflect how structural variations influence global connectivity.

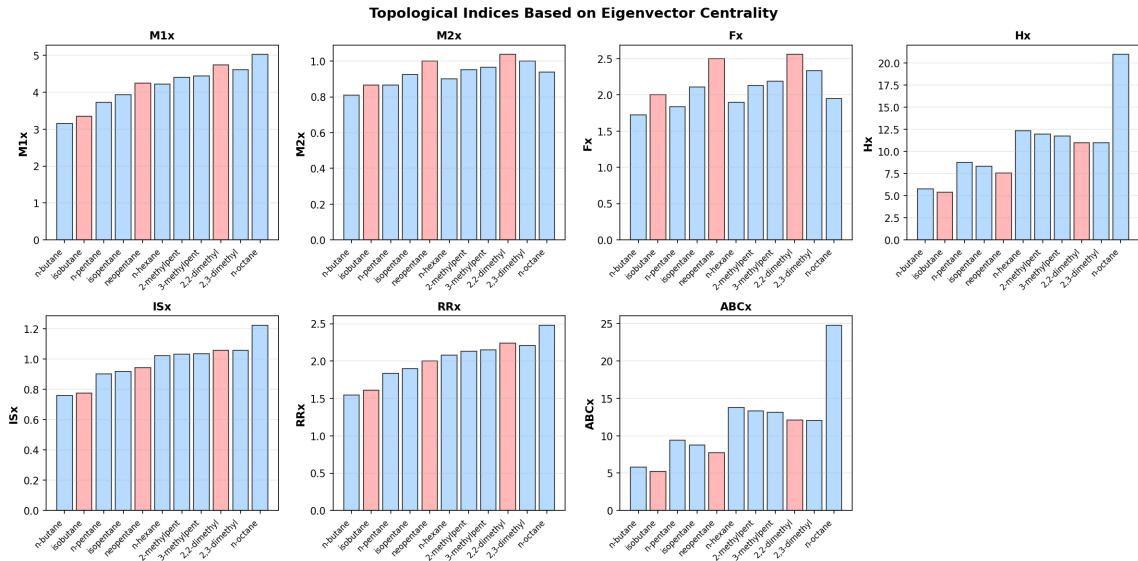


Figure 3: Topological indices derived from eigenvector centrality for alkane isomers.

10.8 Visualization of Centrality Profiles

Figure 4 shows the distribution of eigenvector centrality across atoms for selected alkane isomers.

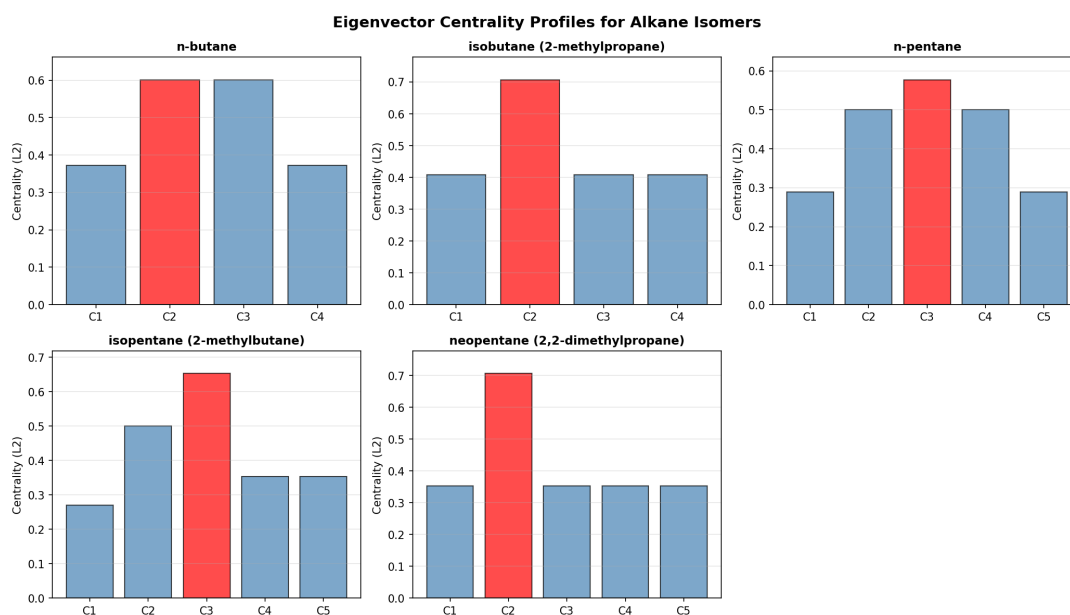


Figure 4: Eigenvector centrality distribution for alkane isomers. Central atoms in branched molecules exhibit significantly higher values.

10.9 Correlation with Physical Properties

To evaluate the effectiveness of eigenvector-based indices, their correlation with boiling point is analyzed.

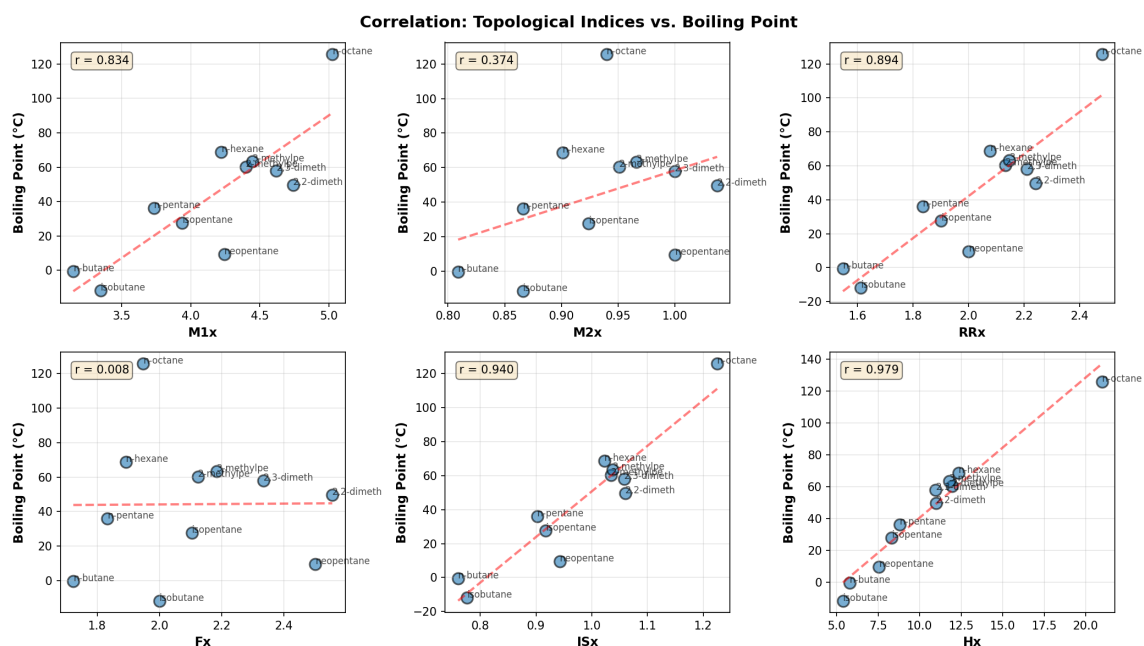


Figure 5: Correlation between topological indices and boiling point for alkane isomers.

Observation:

- Strong positive correlations are observed for indices such as H_x and RR_x (e.g., $r \approx 0.98$ for H_x)
- Moderate correlation is observed for $M1_x$
- Weak or negligible correlation is observed for F_x

Interpretation:

These results directly support the central objective of the project: evaluating eigenvector centrality as a meaningful molecular descriptor.

- Indices such as H_x and RR_x , which depend on interactions between connected nodes, capture **global structural properties** of the molecule
- These indices reflect how centrality is distributed across the graph, which is influenced by branching and connectivity
- Since boiling point depends on molecular shape and intermolecular interactions, the strong correlation indicates that eigenvector-based indices encode physically relevant structural information

In contrast:

- The index F_x , which depends only on squared centrality values, captures more **local or isolated properties**
- This results in poor correlation with boiling point, highlighting that not all centrality-derived measures are equally informative

10.10 Relation to Project Goals

This analysis demonstrates that:

- Eigenvector centrality is not only theoretically valid (under Euclidean normalization), but also **practically useful**
- It can be used to derive topological indices that correlate strongly with real-world physical properties
- Structural features such as branching are effectively captured through centrality distribution

Thus, the project successfully connects:

Graph Structure $\rightarrow$ Eigenvector Centrality $\rightarrow$ Topological Indices $\rightarrow$ Physical Properties

11 Tensor Extension of Centrality

Standard eigenvector centrality considers only direct neighbors. However, tensor-based extensions incorporate higher-order relationships.

Instead of a matrix A_{ij} , a tensor T_{ijk} is used to represent interactions involving paths of length two.

This allows:

- Inclusion of second-order neighbor effects
- Better representation of structural influence
- Application to complex systems such as multilayer networks

In molecular terms, this means that an atom’s importance depends not only on its immediate neighbors but also on neighbors of neighbors.

12 Conclusion

This project demonstrates that:

- Eigenvector centrality provides a global measure of node importance
- It becomes a valid node-centrality only under Euclidean normalization
- Molecular structures can be effectively analyzed using graph-based spectral methods
- Branching significantly affects centrality distribution and derived indices

The results highlight the potential of spectral graph theory in chemical modeling and open avenues for further exploration using tensor-based approaches.

References

- [1] B. Ruhnau, *Eigenvector-centrality — a node-centrality?*, Social Networks, 22(4), 357–365, 2000.
- [2] A. S. J. Rani and B. J. Balamurugan, *Novel eigenvector centrality indices for octane isomers to explore their physicochemical properties*, Scientific Reports, 15, 34730, 2025.
- [3] Two-steps eigenvector centrality in complex networks, Information Sciences, Elsevier, 2023. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0020025526001842>
- [4] NetworkX Developers, *NetworkX Documentation: Eigenvector Centrality*, Available at: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.centrality.eigenvector_centrality.html
- [5] D. Cvetković, M. Doob, and H. Sachs, *Spectra of Graphs: Theory and Application*, Academic Press, 1995.